

BAYESIAN OPTIMIZATION VIA CONTINUAL VARIATIONAL LAST LAYER TRAINING

Paul Brunzema¹

Mikkel Jordahn²

John Willes³

Sebastian Trimpe¹

Jasper Snoek⁴

James Harrison⁴

¹RWTH Aachen University, ²Technical University of Denmark, ³Vector Institute, ⁴Google DeepMind

ABSTRACT

Gaussian Processes (GPs) are widely seen as the state-of-the-art surrogate models for Bayesian optimization (BO) due to their ability to model uncertainty and their performance on tasks where correlations are easily captured (such as those defined by Euclidean metrics) and their ability to be efficiently updated online. However, the performance of GPs depends on the choice of kernel, and kernel selection for complex correlation structures is often difficult or must be made bespoke. While Bayesian neural networks are a promising direction for higher capacity surrogate models, they have so far seen limited use due to a combination of cost of use and poor performance. In this paper, we propose an approach which offers the strengths of both methods. We build on variational Bayesian last layers (VBLLs), which provide a simple and computationally lightweight approach to Bayesian uncertainty quantification in neural networks. We connect training of these models to exact conditioning in GPs, and propose an efficient online training algorithm that interleaves conditioning and optimization. Our findings suggest that VBLL networks significantly outperform GPs and other BNN architectures on tasks with complex input correlations, and match the performance of well-tuned GPs on established benchmark tasks.

1 INTRODUCTION

Bayesian optimization (BO) has become an immensely popular method for optimizing black-box functions that are expensive to evaluate, and has seen large success in a variety of applications (Garnett et al., 2010; Snoek et al., 2012; Calandra et al., 2016; Marco et al., 2016; Frazier & Wang, 2016; Berkenkamp et al., 2017; Chen et al., 2018; Neumann-Brosig et al., 2019; Griffiths & Hernández-Lobato, 2020; Colliandre & Muller, 2023). In BO, the goal is to optimize some black-box objective $f: \mathcal{X} \rightarrow \mathbb{R}^K$ (where $\mathcal{X} \subseteq \mathbb{R}^D$) in as few samples as possible whilst only having access to sequentially sampled, potentially noisy, data points from possibly multiple objectives.

Gaussian processes (GPs) have long been the de-facto surrogate models in BO due to their well-calibrated uncertainty quantification and strong performance in small-data regimes. However, their application becomes challenging in high-dimensional, non-stationary, and structured data environments such as drug-discovery (Colliandre & Muller, 2023; Griffiths & Hernández-Lobato, 2020) and materials science (Frazier & Wang, 2016). Here, often prohibitively expensive or bespoke kernels are necessary to capture meaningful correlations between data points. Furthermore, the scaling of GPs to large datasets typically associated with high-dimensional spaces can be limiting—especially if combined with online hyperparameter estimation. To address these challenges, integrating Bayesian Neural Networks (BNNs) into BO as alternative surrogate models has gained increasing attention (Snoek et al., 2015; Springenberg et al., 2016; Foldager et al., 2023; Li et al., 2024). While BNNs inherently scale with data, challenges like efficiently conditioning on new data and consistency across tasks persist. Moreover, the performance of BNNs on many tasks has so far not matched the performance of GPs due to issues like under-fitting (Li et al., 2024; Wenzel et al., 2020; Ovadia et al., 2019; Izmailov et al., 2021).

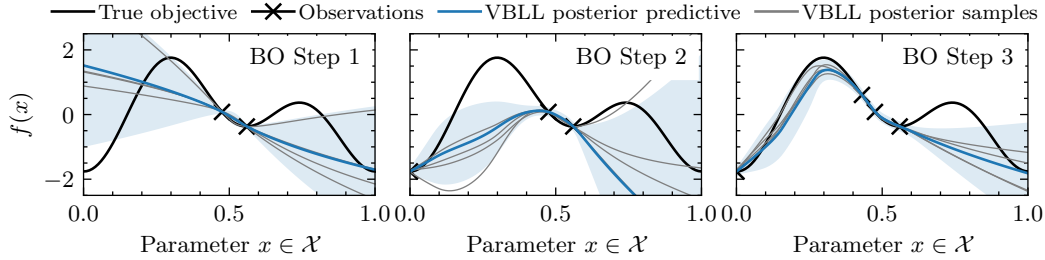


Figure 1: Variational Bayesian last layer model as a surrogate model for BO on a toy example. The VBLL model can capture *in-between* uncertainty and analytic posterior samples are easily obtained through its parametric form making it a suitable surrogate for BO.

In this work, we develop an approach for surrogate modeling in Bayesian optimization that combines the strengths of GPs (efficient conditioning, strong predictive performance, simple and effective uncertainty quantification) with the strengths of BNNs (scalability and ability to handle highly non-Euclidean correlation structures). Specifically, our approach builds on Variational Bayesian Last Layer (VBLL) neural networks (Harrison et al., 2024), which are similar to *parameteric* GPs with a learned kernel.

Contributions: Our main contributions and findings are:

- We show our VBLL surrogate model can outperform a wide variety of baselines on a diverse set of problems, including those with discrete inputs and multi-objective problems.
- We present a connection between model conditioning (in a Bayesian sense) and model optimization, and develop an efficient online optimization scheme that exploits this connection.
- We experimentally connect the hyperparameters of the VBLL model to problem features, such as stochasticity, and discuss implications for other BNN models.
- We release an easy-to-use codebase (link removed for anonymity) to enable use of VBLL models within BoTorch (Balandat et al., 2020).

2 RELATED WORK AND BACKGROUND

Various forms of Bayesian or partially-Bayesian neural networks have been explored for BO, including mean field BNNs (Foldager et al., 2023), networks trained via stochastic gradient Hamiltonian Monte Carlo (Springenberg et al., 2016; Kim et al., 2021), and last layer Laplace approximation BNNs (Kristiadi et al., 2021; Daxberger et al., 2021). Li et al. (2024) find that infinite-width BNNs (I-BNNs) (Lee et al., 2017; Adlam et al., 2021; 2023) perform particularly well especially on high-dimensional, non-stationary and non-Euclidean problems, a setting for which standard GP priors are inappropriate or not well specified, and for which designing suitable kernels is difficult.

While BNNs are promising, they have often proven to be challenging to train and complex to use in practice. Bayesian last layer networks—which consider uncertainty only over the output layer—provide a simple (and often much easier to train) partially-Bayesian neural network model (Snoek et al., 2015; Azizzadenesheli et al., 2018; Harrison et al., 2018; Weber et al., 2018; Riquelme et al., 2018; Fiedler & Lucia, 2023). Concretely, the standard model for regression with Bayesian last layer networks and a one-dimensional¹ output is

$$y = \mathbf{w}^\top \phi_\theta(\mathbf{x}) + \varepsilon, \quad (1)$$

where $\mathbf{w} \in \mathbb{R}^m$ is the last layer for which Bayesian inference is performed, and ϕ_θ are features learned by a neural network backbone with (point estimated) parameters θ . The noise $\varepsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2)$ is assumed to be independent and identically distributed.

¹For multi-variate modeling, we assume each element of ε is independent, and so the N -dimensional problem is separable into N independent inference problems with shared features ϕ_θ . Relaxing this assumption is possible and leads to a matrix normal-distributed posterior.

With this observation model, fixed features ϕ_θ , and a Gaussian prior on the weights as $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}_0, S_0)$, posterior inference for the weights is analytically tractable via Bayesian linear regression. In particular, Bayesian linear regression is possible via a set of recursive updates in the natural parameters

$$\mathbf{q}_t = \sigma^{-2} \phi_t y_t + \mathbf{q}_{t-1} \quad (2)$$

$$S_t^{-1} = \sigma^{-2} \phi_t \phi_t^\top + S_{t-1}^{-1} \quad (3)$$

where $\bar{\mathbf{w}}_t = S_t \mathbf{q}_t$ and $\phi_t := \phi_\theta(\mathbf{x}_t)$. For dataset $\mathcal{D}_T := \{(\mathbf{x}_t, y_t)\}_{t=1}^T$, this recursive update yields posterior $p(\mathbf{w} \mid \mathcal{D}_T) = \mathcal{N}(\bar{\mathbf{w}}_T, S_T)$ and posterior predictive

$$p(y \mid \mathbf{x}, \mathcal{D}_T, \theta) = \mathcal{N}(\bar{\mathbf{w}}_T^\top \phi_\theta(\mathbf{x}), \phi_\theta(\mathbf{x})^\top S_T \phi_\theta(\mathbf{x}) + \sigma^2) \quad (4)$$

Since the predictive distribution is Gaussian, it pairs straight-forwardly with conventional acquisition functions in Bayesian optimization and bandit tasks (Snoek et al., 2015; Riquelme et al., 2018).

Usually, such BLL models are trained using gradient descent on the exact (log) marginal likelihood over all data points. This can be computed either via the marginal likelihood or by backpropagating through the recursive last layer update (Harrison et al., 2018). The cost of this is prohibitive, as it requires iterating over the full dataset for each gradient computation. Moreover, it often is numerically unstable and can result in pathological behavior of the learned features (Thakur et al., 2020; Ober et al., 2021). An alternative strategy is to train on mini-batches (Snoek et al., 2015) to learn features and condition the last layer on the full dataset after the features are trained. However, this yields biased gradients and often results in a severely over-concentrated final model.

To increase efficiency, recent work (Harrison et al., 2024; Watson et al., 2021) developed a deterministic variational lower bound to the exact marginal likelihood and proposed to optimize this instead, resulting in the *variational* Bayesian last layer (VBLL) model. In our setup, we fix a variational posterior $q(\mathbf{w}) = \mathcal{N}(\bar{\mathbf{w}}, S)$. The lack of subscript denotes that the last layer parameters belong to the variational posterior, and we will write $\boldsymbol{\eta} := (\bar{\mathbf{w}}, S)$ for convenience. Following Harrison et al. (2024, Theorem 1), the variational lower bound for regression with BLLs (under the prior defined previously) is

$$\log p(Y \mid X, \theta) \geq \sum_t^T \left(\log \mathcal{N}(y_t \mid \bar{\mathbf{w}}^\top \phi_t, \sigma^2) - \frac{1}{2} \phi_t^\top S \phi_t \sigma^{-2} \right) - \text{KL}(q_\eta(\mathbf{w}) \parallel p(\mathbf{w})) =: \mathcal{L}(\boldsymbol{\eta}, \theta). \quad (5)$$

The variational posterior over the last layer is trained with the network weights θ in a standard neural network training loop, yielding a lightweight Bayesian formulation.

3 TRAINING AND MODEL SPECIFICATION

In this section, we discuss the training procedure we use for VBLLs in the BO loop. We first identify a relationship between variational training of Bayesian last layer models (as is used in the VBLL class of models) and recursive last layer computation (as is used in standard BLL models). We then describe the feature training procedure, and details on early stopping and previous model re-use. Finally, we present the overall training loop that combines these two training approaches, and relevant hyperparameters. We expand on all design decisions in the Appendix.

3.1 VARIATIONAL INFERENCE AND RECURSIVE LAST LAYER UPDATING

In this subsection, we describe a connection between the VBLL variational objective (5)—which is optimized through standard mini-batch optimization—and the recursive updating associated with Bayesian linear regression and BLL models.

Theorem 1. *Fix θ . Then, the variational posterior parameterized by*

$$(\bar{\mathbf{w}}^*, S^*) := \boldsymbol{\eta}^* = \arg \max_{\boldsymbol{\eta}} \mathcal{L}(\boldsymbol{\eta}, \theta) \quad (6)$$

is equivalent to the posterior computed by the recursive least squares inferential procedure described by (2) and (3), iterated over the full dataset.

The proof of this result is provided in the Appendix. This equivalence provides a bridge between two different interpretations of model training: optimization-based training (as is used in NNs) and conditioning (as is used in GPs). This equivalence allows us to consider an online optimization procedure that interleaves two steps: full model training (including feature weights θ) on the variational objective, and last layer-only training via recursive conditioning. While the former optimizes all parameters and the latter only optimizes the last layer parameters, they are optimizing the same objective which stabilizes the interleaving of these steps.

We parameterize a dense precision (in contrast to standard VBLL models which parameterized the covariance) for the variational posterior by Cholesky decomposition, $S^{-1} = LL^\top$ (with L lower triangular). The recursive precision update is then

$$L_t L_t^\top = \phi_t \phi_t^\top + L_{t-1} L_{t-1}^\top. \quad (7)$$

The updated Cholesky factor L_t can be computed efficiently via a rank-1 update that preserves triangularity (Krause & Igel, 2015). The mean update can be computed via the natural parameterization

$$q_t = \phi_t y_t + q_{t-1} \quad (8)$$

from which the last layer mean can be computed. We do not store the computation graph associated with these updates and do not backpropagate through model updates.

Computational complexity: The update formula for the last layer (defined by the rank-1 Cholesky update for the covariance and (8)) has quadratic computational complexity in the feature dimension. Computing $\bar{w}_t$ can similarly be computed with quadratic complexity as the product $L_t^{-\top} L_t^{-1} q_t$. This can be computed via efficient linear solves due to the triangularity of L_t . We also note that computing the log determinant $-\log \det S_t$ is equivalent to computing $\log \det S_t^{-1} = 2 \log \det L_t$, which is equal to the sum of the log diagonal elements, and thus has linear complexity. However, the KL penalty on the last layer variational posterior requires computing the trace of the covariance. This can be expressed as $\|L_t^{-1}\|_F^2$, which can not be computed in quadratic time. With the exception of this one step, the computational complexity is quadratic, but the overall complexity is dominated by computing this trace term. Thus, the complexity is equivalent to that of inverting a triangular matrix—sub-cubic but greater than quadratic. If the complexity of this step is limiting, an approximate method for the computation of this norm (for example, inverse iteration) can be used.

3.2 FULL MODEL TRAINING

In full model training, we directly train the neural network weights θ , the variational posterior parameters η , and the MAP estimate of the noise covariance σ^2 via gradient descent on (5). We decrease training time via early stopping, and further accelerate training via re-using the learned features from a previous full model training step. Both are described below.

Early stopping: The training process for BNNs typically follows standard neural network procedures and relies on a held-out validation set to determine when to stop training (Watson et al., 2021; Harrison et al., 2024). However, in the context of BO with limited data, this approach is impractical, particularly at the early stages of optimization when the training data size is very small. To address this, we implement early stopping for VBLL networks based on the training loss. Specifically, we track the average loss for each training epoch and stop if the average loss does not improve for M consecutive epochs, using the model parameters that produced the lowest training loss.

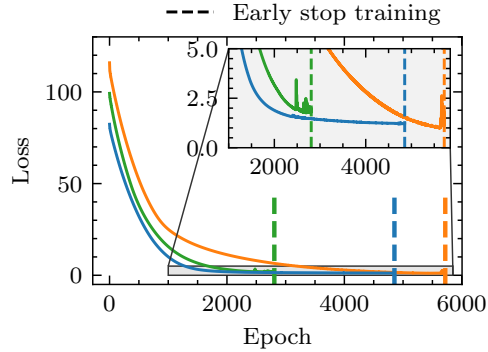


Figure 2: Training loss with early stopping ($M = 100$) for three different model initialization.

Figure 2 shows the training of the same VBLL network with different initialization but using identical data and optimizer settings. It can be seen that the training loss reaches a plateau at significantly different epochs across these initializations,

Algorithm 1 VBLL Bayesian Optimization Loop with Continual Variational Last Layer Training

Require: Model train frequency T_{model} and total number of evaluations T ; Wishart prior scale V

- 1: $\mathcal{D} \leftarrow \mathcal{D}_{\text{init}}$ obtained from evaluations at a pseudo-random input locations from a Sobol sequence
- 2: **for** $t = 0$ to T **do**
- 3: **if** $t \bmod T_{\text{model}} = 0$ **then**
- 4: Init. model parameters η, θ, σ^2 and train via minibatch optimization on (5) using $\mathcal{D}$
- 5: **else**
- 6: Update η via (rank-1 Cholesky) recursive updates (2) and (3) using $(\mathbf{x}_{t-1}, y_{t-1})$ and Σ
- 7: **end if**
- 8: Select $\mathbf{x}_t$ via optimizing acquisition function; query objective function with $\mathbf{x}_t$ and receive y_t
- 9: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{x}_t, y_t)\}$
- 10: **end for**

as indicated by the dashed vertical lines. Such a large variance in the optimal number of training epochs was also observed in the original VBLL paper for regression tasks (Harrison et al., 2024). By applying an early stopping criterion, we are able to specify a high maximum number of epochs while still reducing computational costs, as each initialization is only trained *as long as necessary* to achieve optimal performance.

It is important to emphasize that this approach is not exclusive to VBLLs; we believe that applying a similar early stopping criterion could prove beneficial for training neural network-based surrogate models in BO more broadly. In our experiments, we therefore applied early stopping to all applicable BNN baselines.

Feature re-use: The naive approach to training requires training a new network for each step of full model training. While we mitigate the cost of training via the last layer recursive update, we further explore using continual feature learning for faster convergence. For this, we initialize the VBLLs at each (full model training) iteration with the feature weights from the last full model training iteration, and the variational posterior from the previous recursive update iteration. This warm start, combined with early stopping, yields substantial speed-ups in training. This feature re-use approach follows conventional continual learning, in which networks are trained for streaming data (Zenke et al., 2017; Hadsell et al., 2020). However, as discussed in Appendix B, re-initializing networks plays an important role that has strong connections to Thompson sampling over the features, and thus we will interleave continual (feature) training with occasional model re-initialization.

3.3 FULL TRAINING LOOP AND HYPERPARAMETERS

Our full train loop (Algorithm 1) interleaves two training steps: full model training via minibatch gradient descent, and last layer conditioning. We choose to alternate between these steps at a fixed frequency, doing full model training one out of every T_{model} steps. It may be possible to adaptively choose whether to re-train the model based on features of the model training or prediction, but we found fixed frequency re-training was effective and, importantly, easy to tune. While Algorithm 1 does not explicitly mention feature re-use, we also investigate warm-starting optimization on a fixed schedule.

Optimization: In both newly initialized features and warm-starting, we will re-initialize the optimizer state. Because we are interleaving recursive updates, continued use of previous optimizer states such as momentum and scale normalization (as in e.g. Adam (Kingma & Ba, 2015)) actively harm performance. However, we do find benefits in using adaptive optimizers for training in general and use AdamW (Loshchilov & Hutter, 2017) throughout our experiments (with no regularization on the variational last layer).

Hyperparameters: The VBLL models have several hyperparameters, some of which are similar to GPs and some of which are substantially different. We include hyperparameter studies in Appendix D. We specifically investigate the hyperparameter sensitivity of the noise covariance prior, network reinitialization rate and neural network architecture. Overall we find that the VBLL models are highly robust to these hyperparameters, but that proper tuning of them for the problem at hand can further improve performance.

4 ACQUISITION FUNCTIONS

In this section we describe acquisition functions for our VBLL models in both the single objective and multi-objective case.

4.1 SINGLE OBJECTIVE ACQUISITION FUNCTIONS

VBLLs yield a Gaussian predictive distribution and thus most acquisition functions that are straightforward to compute for GPs are also straightforward for VBLLs. However, parametric VBLLs are also especially well suited for Thompson sampling compared to non-parametric models like GPs². For Thompson sampling, we simply sample from the variational posterior of $\mathbf{w}$ at iteration k and then construct a sample from the predictive $\hat{f}$ (cf. Fig. 1) as a generalized linear model as

$$\textcircled{1} \quad \hat{\mathbf{w}} \sim q_{\boldsymbol{\eta}}^k(\mathbf{w}) \quad \textcircled{2} \quad \hat{f}(\mathbf{x}) := \hat{\mathbf{w}}^\top \boldsymbol{\phi}_{\boldsymbol{\theta}}(\mathbf{x}) \quad \textcircled{3} \quad \mathbf{x}_{k+1} = \arg \max_{\mathbf{x} \in \mathcal{X}} \hat{f}(\mathbf{x}) \quad (9)$$

This sample of the predictive can then be optimized *analytically*, which differs from classic Thompson sampling methods used for non-parametric GPs. This can similarly be done for LLLA BNNs (cf. Sec. 5), although Laplace approximated posterior samples are known to suffer from underfitting on training data (Lawrence, 2001; Roy et al., 2024). For the analytic optimization of the parametric sample, we use L-BFGS-B (Zhu et al., 1997) leveraging the fact that we can also easily obtain the gradient of $\hat{f}$. We initialize the optimization at multiple random initial conditions³ and choose the best argmax as the next query location.

4.2 MULTI-OBJECTIVE ACQUISITION FUNCTIONS

In multi-objective BO it is common to model each objective with a separate GP (Zitzler et al., 2003; Daulton et al., 2020; Ament et al., 2024). While it is often preferable to model correlations between outputs (Swersky et al., 2013), this requires correlation kernels for the objectives, which may difficult to specify. With neural network based surrogate models, one can simply set the number of outputs to the number of objectives, thus sharing feature learning between objectives. For VBLL networks, we can use the multi-variate regression formulation to obtain a multivariate normal prediction for each point. Since this again is a Gaussian, it is straightforward to use popular acquisition functions for multi objective optimization such as expected hypervolume improvement (Zitzler et al., 2003; Daulton et al., 2020; Ament et al., 2024).

For multi-objective optimization problems, we can leverage the parametric form of the VBLLs to do efficient Thompson sampling. It is also possible to do Thompson sampling with GPs for multi-objective optimization, but this usually involves approximations with a high number of Fourier bases functions (Bradford et al., 2018; Belakaria et al., 2019; Paria et al., 2020). For Thompson sampling with VBLLs, we first sample a NN realization of the VBLL network as in (9) and analytically optimize the sample with NSGA-II (Deb et al., 2002) using Pymoo (Blank & Deb, 2020). This yields a predicted Pareto front $\hat{\mathcal{P}} := \{\hat{\mathbf{y}}_1, \dots, \hat{\mathbf{y}}_P\}$ with size equal to the population size P of the NSGA-II solver. In summary, the Thompson sampling procedure for multi-objective optimization is

$$\textcircled{1} \quad \hat{W} \sim q_{\boldsymbol{\eta}}^k(W) \quad \textcircled{2} \quad \hat{f}(\mathbf{x}) := \hat{W}^\top \boldsymbol{\phi}_{\boldsymbol{\theta}}(\mathbf{x}) \quad \textcircled{3} \quad \hat{\mathcal{P}} = \max_{\mathbf{x} \in \mathcal{X}} \hat{f}(\mathbf{x}). \quad (10)$$

To choose the next candidate, we greedily select the index of the point in the predicted Pareto front that maximizes the improvement of the hypervolume. The hypervolume, denoted by $\mathcal{HV}(\mathcal{P}, \mathbf{r})$, is a commonly used metric in multi-objective optimization, which measures the volume of the region dominated by the Pareto front $\mathcal{P}$ with respect to a predefined reference point $\mathbf{r}$ in the objective space. Our goal in multi-objective BO is to expand this dominated region by adding a new candidate to

²Thompson sampling for GPs often involves drawing samples from high-dimensional posterior distributions generated at pseudo-random input locations (e.g., using Sobol sequences) and then selecting the argmax of the discrete samples as the next query locations (Eriksson et al., 2024). It is worth noting that while it is possible to construct analytic approximate posterior samples for GPs (Wilson et al., 2020; 2021), this approach is not yet commonly adopted in current practice, and is not possible to do for all kernels.

³In the following, the number of random initial conditions will be set to the same number of random initial conditions as for the standard optimization of the acquisition functions for a fair comparison.

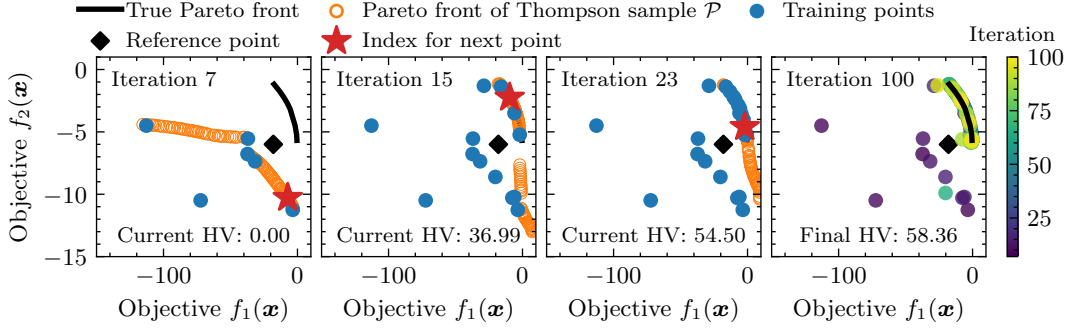


Figure 3: *Multi-objective Thompson sampling on BraninCurrin*. At each iteration, we optimize the multi-objective Thompson sample and choose as the next point the index that increases the predicted hypervolume the most. At the end of the optimization (right with colormap), we can observe that the true Pareto front is nicely approximated.

the Pareto front. At each iteration k , the current Pareto front is denoted by $\mathcal{P}_k$. To select the next candidate, we maximize the hypervolume improvement, which is the increase in hypervolume when adding a predicted new point $\hat{\mathbf{y}} \in \hat{\mathcal{P}}$ to the existing Pareto front $\mathcal{P}_k$ as

$$(4) \quad \mathbf{x}_{k+1} \in \arg \max_{\hat{\mathbf{y}} \in \hat{\mathcal{P}}} \mathcal{HV}(\mathcal{P}_k \cup \{\hat{\mathbf{y}}\}, \mathbf{r}). \quad (11)$$

If none of points in the predicted Pareto front improve the hypervolume or the solution is non-unique, we randomly sample from the set of maximizers. Figure 3 shows Thompson sampling with VBLLs on the BraninCurrin benchmark (cf. Sec. 5). After the initialization, none of the points in the Pareto front $\hat{\mathcal{P}}$ improve the HV. Therefore a random index from the list of solutions in $\hat{\mathcal{P}}$ is chosen for the next query location. After some iteration, the $\hat{\mathcal{P}}$ partially includes the true Pareto front. The algorithm continuously samples points that improve the HV until after 100 iterations $\mathcal{P}$ is well approximated.

In all experiments in this paper, we assume the reference point to be known but we should note that it is also possible to estimate or adaptively choose the reference point $\mathbf{r}$, which can improve the effectiveness of the optimization process (Bradford et al., 2018). Additionally, this strategy of maximizing hypervolume can be combined with other acquisition functions; of particular interest are information-theoretic acquisition functions as in the single objective case. For instance, max-value entropy search for multi-objective optimization (Belakaria et al., 2019) relies on random Fourier features to approximate samples from the GP posterior; this can easily be switched to using Thompson sampling with VBLL networks.

5 RESULTS

We evaluate the performance of the VBLL surrogate model on various standard benchmarks and three more complex optimization problems, where the optimization landscape is non-stationary. For experimental details and ablations of hyperparameters, we refer the reader to Appendix C and D.

5.1 SURROGATE MODELS

We use the following baselines throughout the experiments in addition to VBLLs.

GPs: As the de-facto standard in BO, we compare against GPs. As kernel, we choose a Matérn kernel with $\nu = 2.5$ and use individual lengthscales ℓ_i for all input dimensions that are optimized within box constraints following recommended best practices (Eriksson et al., 2024; Balandat et al., 2020). We use box constraints as $\ell_i \in [0.005, 4]$ (Eriksson et al., 2024). We expect the performance of GPs to be particularly good on stationary benchmarks.

I-BNNs: We compare against infinite-width Bayesian neural networks (I-BNNs) (Lee et al., 2018), which have shown promising results in recent work (Li et al., 2024). As in Li et al. (2024), we set the

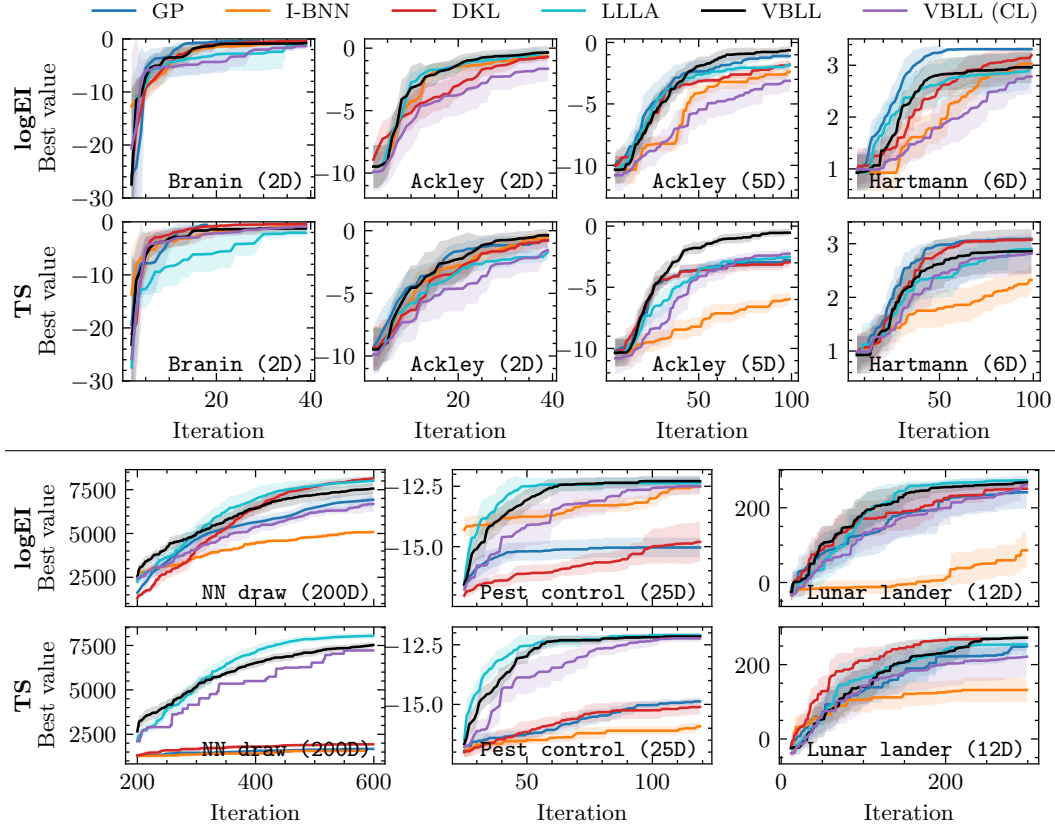


Figure 4: *Classic benchmarks (top) and high-dimensional and non-stationary benchmarks (bottom).* Performance of all surrogates for $\log EI$ (top) and TS (bottom).

depth to 3 and initialize the weight variance to 10 and the bias variance to 1.6. Note that the I-BNN is expressed as a kernel and therefore the model is still non-parametric and does not learn features.

DKL: Deep kernel learning (DKL) (Wilson et al., 2016) combines feature extracting with neural networks with GPs. It uses a neural network g_θ as input wrapping to the kernel as $k_{DKL} := k(g_\theta(\mathbf{x}), g_\theta(\mathbf{x}'))$ to allow for non-stationary modeling and exact inference. For the neural network, we use the same architecture as in (Li et al., 2024), i.e., 3 layers with 128 neurons. We further use ELU activations for all layers.

LLLA: Last layer Laplace approximations (LLLA) are a computationally efficient way obtain uncertainty estimates *after* training a neural network (MacKay, 1992; Daxberger et al., 2021). With this, they are a well suited BNN surrogate model for BO (Kristiadi et al.; Li et al., 2024). As NN, we also use 3 layers with 128 neurons and ELU activations. Note that for TS, we can also optimize the parametric neural network function analytically with LLLA, which is a novel contribution as well.

VBLL: For the VBLLs, we use the same architecture as for DKL and LLLA. In the body of the paper, we include two VBLL models: training the features from scratch every iteration (performant but expensive) and re-training the features every 5 steps and otherwise doing recursive last layer updates. Alternative choices for these training hyperparameters are included in the Appendix.

In all subsequent experiments, we select the number of initial points for the single objective benchmarks equal to the input dimensionality D and for the multi-objective benchmarks we use $2 \cdot (D + 1)$ initial points (Daulton et al., 2020; Balandat et al., 2020). We further use a batch size of one as in the classic BO setting.⁴ We compare the performance of all surrogates for the following acquisition

⁴Note that this differs from the experimental setup by Li et al. (2024) which use larger batch sizes.

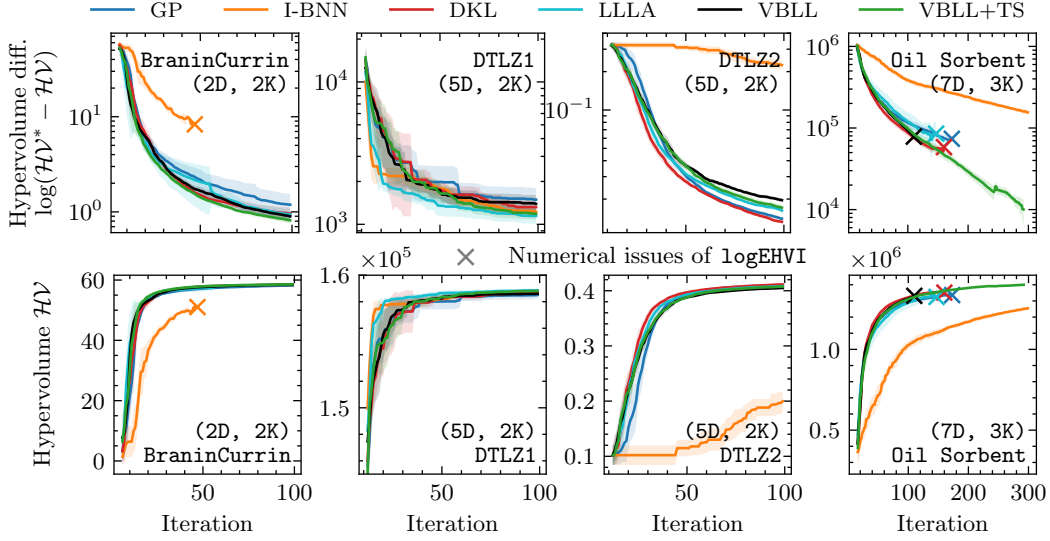


Figure 5: *Multi-objective benchmarks.* Performance of all surrogate models using $\log\text{EHVI}$ and VBLLs with TS. A cross indicates the crash of a surrogate’s furthest seed due to a numerically unstable acquisition function. VBLL+TS successfully navigates the numerically unstable HV plateau in OilSorbent, enabling a more accurate approximation of its three-dimensional Pareto front.

functions: (i) log expected improvement ($\log\text{EI}$) (Ament et al., 2024), a numerically more stable version of standard expected improvement, (ii) Thompson sampling (TS) (Thompson, 1933; Russo et al., 2018), (iii) and log expected hypervolume improvement $\log\text{EHVI}$ (Ament et al., 2024) for the multi-objective benchmarks.

5.2 BENCHMARK PROBLEMS

We begin by examining a set of standard benchmark problems commonly used to assess the performance of BO algorithms (Eriksson et al., 2024; Ament et al., 2024). Figure 4 (top) illustrates the performance of all surrogates on these benchmark problems. It can be observed that, as expected, GPs perform well. The BNN baselines also demonstrate strong performance on lower-dimensional problems, although they do not match the performance of GPs on the Hartmann function. Interestingly, for TS, we notice that on the Ackley5D benchmark, the VBLLs with analytic optimization of the Thompson samples even surpass the performance of GPs as well as all other baselines. The continual learning baselines shows slightly worse performance as the standard VBLLs but at the benefit of reduced compute.

5.3 HIGH-DIMENSIONAL AND NON-STATIONARY PROBLEMS

GPs without tailored kernels often struggle in high-dimensional and non-stationary environments (Li et al., 2024); areas where deep learning approaches are expected to excel. Our results on the 200D NNdraw benchmark (Li et al., 2024), the real-world 25D Pestcontrol benchmark (Oh et al., 2019), and the 12D Lunarlander benchmark (Eriksson et al., 2024) are shown in Figure 4 (bottom). On these benchmarks, all BNN surrogates show strong performance; especially LLLA and the VBLL networks. While GPs perform well with $\log\text{EI}$ on NNdraw and the I-BNNs show good performance on Pestcontrol, the VBLLs and LLLA models are consistently the best performing surrogates. Similar to the classic benchmarks, the continual learning version of the VBLLs shows similar performance to the VBLLs.

5.4 MULTI-OBJECTIVE PROBLEMS

Lastly, we evaluate the VBLL networks on multi-objective optimization problems. Here, we consider the standard benchmarks BraninCurrin ($D = 2, K = 2$), DTLZ1 ($D = 5, K = 2$), DTLZ2 ($D = 5, K = 2$), and the real world benchmark Oil Sorbent ($D = 7, K = 3$) (Wang et al., 2020; Li et al., 2024). Figure 5 shows the performance of all surrogate regarding the obtained HV with respect to fixed reference points and the logarithmic HV difference between the maximum HV⁵ and the obtained HV as common metrics in multi-objective BO (Belakaria et al., 2019; Daulton et al., 2020). The performance of all BNN models is similar. It is however notable that on OilSorbent all baselines using $\log_{\text{EHVI}}$ crash due to numerical issues while optimizing the acquisition function. Such numerical issues are known for expected improvement type acquisition functions due to vanishing gradients (Ament et al., 2024). The Thompson sampling with VBLLs does not results in such numerical issues and we can observe that with this combination of model and acquisition function the Pareto front can further be refined.

6 DISCUSSION AND CONCLUSIONS

In this paper, we have developed a bridge between classical conjugate Bayesian methods for BO and BNN-based methods. We built upon VBLLs (Harrison et al., 2024) to obtain the expressivity of neural networks, together with the simple and robust uncertainty characterization associated with GPs.

Continual learning: We introduced an online training scheme that enables online complexity comparable to GPs. While we have developed an efficient training scheme, further developments in combining continual learning with our architecture are possible. First, we use a simple fixed frequency to update our network features. It is potentially possible to instead update features based on predictive loss or other measurable features of the problem. Second, we use a simple continual learning scheme for the features to increase training efficiency. While this was effective, many methods in continual learning (e.g. Nguyen et al. (2018)) have been developed to improve training efficiency. While the combination of previously-developed continual learning schemes with our last layer updating may be non-trivial, this is likely a promising direction for improving efficiency. Balancing the clear Thompson sampling-like benefit of network re-initialization with efficient continual learning will likely be a necessary focus of future work in this direction.

Performance and baselines: Our findings show that VBLLs perform on par with GPs on standard low-dimensional benchmarks, yet significantly outperform GPs in high-dimensional and non-stationary problems. Furthermore, VBLLs outperform a wide array of Bayesian neural network models, including I-BNNs (Adlam et al., 2021), Laplace approximation-based models (Daxberger et al., 2021), and deep kernel learning (Wilson et al., 2016). While Laplace-approximation methods perform well in our experiments, we find that they are often more sensitive to noise.

Laplace methods approximate the predictive covariance with the (inverse) Hessian. In the line of work on Laplace approximations for neural networks (e.g. Daxberger et al. (2021)), the Fisher information matrix—which consists of the outer-product of network gradients, summed over all data—is used as an approximation to the Hessian. For last layer Laplace in particular, this reduces to the outer product of features, equivalent to covariance computation for Bayesian last layer methods. The primary difference is Daxberger et al. (2021) also typically optimize the hyperparameters (including the last layer prior and hyperparameters) via an empirical Bayes objective. Thus, the last layer Laplace method in this setting can be interpreted as a form of the (non-Variational) Bayesian last layer model. While we optimize the noise covariance we do not optimize the last layer prior, which is the primary difference between our VBLL-based approach and last layer Laplace approximations.

Acquisition functions: Because of the parametric architecture of VBLLs, they can efficiently be combined with Thompson sampling as an acquisition function. This yields reasonable benefits in the uni-variate case (as shown by our full set of results) but yields substantial benefits in the multi-objective case due to better numerical conditioning. Moreover, the ease of use with Thompson sampling also enable easy use of TS-based acquisition functions such as max-value entropy search (Wang & Jegelka, 2017; Belakaria et al., 2019).

⁵For BraninCurrin, DTLZ1, and DTLZ2 we use the values provided by BoTorch, and for OilSorbent we estimate the maximum HV based on the asymptotic performance of the best performing surrogate.

REFERENCES

- Ben Adlam, Jaehoon Lee, Lechao Xiao, Jeffrey Pennington, and Jasper Snoek. Exploring the uncertainty properties of neural networks’ implicit priors in the infinite-width limit. *International Conference on Learning Representations (ICLR)*, 2021.
- Ben Adlam, Jaehoon Lee, Shreyas Padhy, Zachary Nado, and Jasper Snoek. Kernel regression with infinite-width neural networks on millions of examples. *arXiv:2303.05420*, 2023.
- Sebastian Ament, Samuel Daulton, David Eriksson, Maximilian Balandat, and Eytan Bakshy. Unexpected improvements to expected improvement for Bayesian optimization. *Neural Information Processing Systems (NeurIPS)*, 36, 2024.
- Kamyar Azizzadenesheli, Emma Brunskill, and Animashree Anandkumar. Efficient exploration through Bayesian deep q-networks. *arXiv:1802.04412*, 2018.
- Maximilian Balandat, Brian Karrer, Daniel R. Jiang, Samuel Daulton, Benjamin Letham, Andrew Gordon Wilson, and Eytan Bakshy. BoTorch: A Framework for Efficient Monte-Carlo Bayesian Optimization. In *Neural Information Processing Systems (NeurIPS)*, 2020.
- Syrine Belakaria, Aryan Deshwal, and Janardhan Rao Doppa. Max-value entropy search for multi-objective Bayesian optimization. *Neural Information Processing Systems (NeurIPS)*, 32, 2019.
- Felix Berkenkamp, Matteo Turchetta, Angela Schoellig, and Andreas Krause. Safe model-based reinforcement learning with stability guarantees. *Neural Information Processing Systems (NeurIPS)*, 2017.
- Julian Blank and Kalyanmoy Deb. Pymoo: Multi-objective optimization in python. *IEEE Access*, 8: 89497–89509, 2020.
- Eric Bradford, Artur M Schweidtmann, and Alexei Lapkin. Efficient multiobjective optimization employing Gaussian processes, spectral sampling and a genetic algorithm. *Journal of Global Optimization*, 71(2):407–438, 2018.
- Roberto Calandra, André Seyfarth, Jan Peters, and Marc Peter Deisenroth. Bayesian optimization for learning gaits under uncertainty: An experimental comparison on a dynamic bipedal walker. *Annals of Mathematics and Artificial Intelligence*, 76:5–23, 2016.
- Yutian Chen, Aja Huang, Ziyu Wang, Ioannis Antonoglou, Julian Schrittwieser, David Silver, and Nando de Freitas. Bayesian optimization in alphago. *arXiv:1812.06855*, 2018.
- Lionel Colliandre and Christophe Muller. Bayesian optimization in drug discovery. *High Performance Computing for Drug Discovery and Biomedicine*, 2023.
- Samuel Daulton, Maximilian Balandat, and Eytan Bakshy. Differentiable expected hypervolume improvement for parallel multi-objective Bayesian optimization. *Neural Information Processing Systems (NeurIPS)*, 33:9851–9864, 2020.
- Erik Daxberger, Agustinus Kristiadi, Alexander Immer, Runa Eschenhagen, Matthias Bauer, and Philipp Hennig. Laplace redux-effortless Bayesian deep learning. *Neural Information Processing Systems (NeurIPS)*, 2021.
- Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and TAMT Meyarivan. A fast and elitist multi-objective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2): 182–197, 2002.
- David Eriksson, Michael Pearce, Jacob Gardner, Ryan D Turner, and Matthias Poloczek. Scalable global optimization via local Bayesian optimization. In *Neural Information Processing Systems (NeurIPS)*, 2024.
- Felix Fiedler and Sergio Lucia. Improved uncertainty quantification for neural networks with Bayesian last layer. *IEEE Access*, 2023.
- Jonathan Foldager, Mikkel Jordahn, Lars Kai Hansen, and Michael Riis Andersen. On the role of model uncertainties in Bayesian optimization. In *Uncertainty in Artificial Intelligence (UAI)*, 2023.

- Peter I Frazier and Jialei Wang. Bayesian optimization for materials design. *Information science for materials discovery and design*, 2016.
- Jacob R Gardner, Geoff Pleiss, David Bindel, Kilian Q Weinberger, and Andrew Gordon Wilson. Gpytorch: Blackbox matrix-matrix gaussian process inference with gpu acceleration. In *Neural Information Processing Systems (NeurIPS)*, 2018.
- Roman Garnett, Michael A Osborne, and Stephen J Roberts. Bayesian optimization for sensor set selection. *International conference on information processing in sensor networks*, 2010.
- Ryan-Rhys Griffiths and José Miguel Hernández-Lobato. Constrained Bayesian optimization for automatic chemical design using variational autoencoders. *Chemical science*, 11(2):577–586, 2020.
- Raia Hadsell, Dushyant Rao, Andrei A Rusu, and Razvan Pascanu. Embracing change: Continual learning in deep neural networks. *Trends in cognitive sciences*, 2020.
- James Harrison, Apoorva Sharma, and Marco Pavone. Meta-learning priors for efficient online Bayesian regression. *Workshop on the Algorithmic Foundations of Robotics (WAFR)*, 2018.
- James Harrison, John Willes, and Jasper Snoek. Variational Bayesian last layers. *International Conference on Learning Representations (ICLR)*, 2024.
- Pavel Izmailov, Sharad Vikram, Matthew D Hoffman, and Andrew Gordon Gordon Wilson. What are bayesian neural network posteriors really like? In *International Conference on Machine Learning (ICML)*, 2021.
- Samuel Kim, Peter Y Lu, Charlotte Loh, Jamie Smith, Jasper Snoek, and Marin Soljačić. Deep learning for Bayesian optimization of scientific problems with high-dimensional structure. *arXiv:2104.11667*, 2021.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*, 2015.
- Jeremias Knoblauch, Jack Jewson, and Theodoros Damoulas. Generalized variational inference: Three arguments for deriving new posteriors. *arXiv preprint arXiv:1904.02063*, 2019.
- Oswin Krause and Christian Igel. A more efficient rank-one covariance matrix update for evolution strategies. In *ACM Conference on Foundations of Genetic Algorithms*, 2015.
- Agustinus Kristiadi, Alexander Immer, Runa Eschenhagen, and Vincent Fortuin. Promises and pitfalls of the linearized laplace in Bayesian optimization. In *Fifth Symposium on Advances in Approximate Bayesian Inference*.
- Agustinus Kristiadi, Matthias Hein, and Philipp Hennig. Learnable uncertainty under laplace approximations. In *Uncertainty in Artificial Intelligence (UAI)*, 2021.
- Neil David Lawrence. *Variational inference in probabilistic models*. PhD thesis, Citeseer, 2001.
- Gilwoo Lee, Siddhartha S Srinivasa, and Matthew T Mason. GP-iLQG: Data-driven robust optimal control for uncertain nonlinear dynamical systems. *arXiv:1705.05344*, 2017.
- Jaehoon Lee, Yasaman Bahri, Roman Novak, Samuel S Schoenholz, Jeffrey Pennington, and Jascha Sohl-Dickstein. Deep neural networks as Gaussian processes. *International Conference on Learning Representations (ICLR)*, 2018.
- Yucen Lily Li, Tim G. J. Rudner, and Andrew Gordon Wilson. A study of Bayesian neural network surrogates for Bayesian optimization. In *International Conference on Learning Representations (ICLR)*, 2024.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv:1711.05101*, 2017.
- David JC MacKay. A practical bayesian framework for backpropagation networks. *Neural Computation*, 1992.

- Alonso Marco, Philipp Hennig, Jeannette Bohg, Stefan Schaal, and Sebastian Trimpe. Automatic lqr tuning based on gaussian process global optimization. *IEEE International Conference on Robotics and Automation (ICRA)*, 2016.
- Matthias Neumann-Brosig, Alonso Marco, Dieter Schwarzmann, and Sebastian Trimpe. Data-efficient autotuning with Bayesian optimization: An industrial control study. *IEEE Transactions on Control Systems Technology*, 28(3):730–740, 2019.
- Cuong V Nguyen, Yingzhen Li, Thang D Bui, and Richard E Turner. Variational continual learning. In *International Conference on Learning Representations (ICLR)*, 2018.
- Sebastian W Ober, Carl E Rasmussen, and Mark van der Wilk. The promises and pitfalls of deep kernel learning. In *Uncertainty in Artificial Intelligence (UAI)*, 2021.
- Changyong Oh, Jakub Tomczak, Efstratios Gavves, and Max Welling. Combinatorial bayesian optimization using the graph cartesian product. *Neural Information Processing Systems (NeurIPS)*, 32, 2019.
- Yaniv Ovadia, Emily Fertig, Jie Ren, Zachary Nado, Sebastian Nowozin, Joshua Dillon, Balaji Lakshminarayanan, and Jasper Snoek. Can you trust your model’s uncertainty? evaluating predictive uncertainty under dataset shift. In *Neural Information Processing Systems (NeurIPS)*, 2019.
- Biswajit Paria, Kirthevasan Kandasamy, and Barnabás Póczos. A flexible framework for multi-objective Bayesian optimization using random scalarizations. In *Uncertainty in Artificial Intelligence*, pp. 766–776. PMLR, 2020.
- Carlos Riquelme, George Tucker, and Jasper Snoek. Deep Bayesian bandits showdown. In *International Conference on Learning Representations (ICLR)*, 2018.
- Hritik Roy, Marco Miani, Carl Henrik Ek, Philipp Hennig, Marvin Pförtner, Lukas Tatzel, and Søren Hauberg. Reparameterization invariance in approximate Bayesian inference, 2024.
- Daniel J Russo, Benjamin Van Roy, Abbas Kazerouni, Ian Osband, Zheng Wen, et al. A tutorial on Thompson sampling. *Foundations and Trends in Machine Learning*, 2018.
- Jasper Snoek, Hugo Larochelle, and Ryan P Adams. Practical Bayesian optimization of machine learning algorithms. *Neural Information Processing Systems (NeurIPS)*, 2012.
- Jasper Snoek, Oren Rippel, Kevin Swersky, Ryan Kiros, Nadathur Satish, Narayanan Sundaram, Mostofa Patwary, Prabhat, and Ryan Adams. Scalable Bayesian optimization using deep neural networks. *International Conference on Machine Learning (ICML)*, 2015.
- Jost Tobias Springenberg, Aaron Klein, Stefan Falkner, and Frank Hutter. Bayesian optimization with robust bayesian neural networks. *Neural Information Processing Systems (NeurIPS)*, 2016.
- Kevin Swersky, Jasper Snoek, and Ryan P. Adams. Multi-task bayesian optimization. 2013.
- Sujay Thakur, Cooper Lorusong, Yaniv Yacoby, Finale Doshi-Velez, and Weiwei Pan. Uncertainty-aware (una) bases for Bayesian regression using multi-headed auxiliary networks. *arXiv preprint arXiv:2006.11695*, 2020.
- William R Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 1933.
- Boqian Wang, Jiacheng Cai, Chuangui Liu, Jian Yang, and Xianting Ding. Harnessing a novel machine-learning-assisted evolutionary algorithm to co-optimize three characteristics of an electrospun oil sorbent. *ACS Applied Materials & Interfaces*, 12(38):42842–42849, 2020.
- Zi Wang and Stefanie Jegelka. Max-value entropy search for efficient Bayesian optimization. In *International Conference on Machine Learning (ICML)*, pp. 3627–3635. PMLR, 2017.
- Joe Watson, Jihao Andreas Lin, Pascal Klink, Joni Pajarinen, and Jan Peters. Latent derivative Bayesian last layer networks. In *Artificial Intelligence and Statistics (AISTATS)*, 2021.

- Noah Weber, Janez Starc, Arpit Mittal, Roi Blanco, and Lluís Màrquez. Optimizing over a Bayesian last layer. In *NeurIPS workshop on Bayesian Deep Learning*, 2018.
- Florian Wenzel, Kevin Roth, Bastiaan S Veeling, Jakub Swiatkowski, Linh Tran, Stephan Mandt, Jasper Snoek, Tim Salimans, Rodolphe Jenatton, and Sebastian Nowozin. How good is the bayes posterior in deep neural networks really? In *International Conference on Machine Learning (ICML)*, 2020.
- Andrew Gordon Wilson, Zhiting Hu, Ruslan Salakhutdinov, and Eric P Xing. Deep kernel learning. In *Artificial Intelligence and Statistics (AISTATS)*, 2016.
- James Wilson, Viacheslav Borovitskiy, Alexander Terenin, Peter Mostowsky, and Marc Deisenroth. Efficiently sampling functions from Gaussian process posteriors. In *International Conference on Machine Learning (ICML)*. PMLR, 2020.
- James T Wilson, Viacheslav Borovitskiy, Alexander Terenin, Peter Mostowsky, and Marc Peter Deisenroth. Pathwise conditioning of Gaussian processes. *Journal of Machine Learning Research*, 22(105):1–47, 2021.
- Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. *International Conference on Machine Learning (ICML)*, 2017.
- Ciyu Zhu, Richard H Byrd, Peihuang Lu, and Jorge Nocedal. Algorithm 778: L-bfgs-b: Fortran subroutines for large-scale bound-constrained optimization. *ACM Transactions on mathematical software (TOMS)*, 23(4):550–560, 1997.
- Eckart Zitzler, Lothar Thiele, Marco Laumanns, Carlos M Fonseca, and Viviane Grunert Da Fonseca. Performance assessment of multiobjective optimizers: An analysis and review. *IEEE Transactions on Evolutionary Computation*, 7(2):117–132, 2003.

A	Algorithmic Details	16
A.1	Proof of Theorem 1	16
A.2	Training Details	16
A.3	Further Details on the Setup and Acquisition Functions	16
B	On Neural Network Thompson Sampling	16
C	Experimental Details	18
D	Hyperparameter Sensitivity	18
D.1	Wishart Scale and Continual Learning Sensitivity	18
D.2	Model Width Ablation	20
D.3	Model Performance in the Presence of Noise	22

A ALGORITHMIC DETAILS

A.1 PROOF OF THEOREM 1

We restate the theorem for completeness.

Theorem 1. *Fix θ . Then, the variational posterior parameterized by*

$$(\bar{w}^*, S^*) := \boldsymbol{\eta}^* = \arg \max_{\boldsymbol{\eta}} \mathcal{L}(\boldsymbol{\eta}, \theta) \quad (12)$$

is equivalent to the posterior computed by the recursive least squares inferential procedure described by (2) and (3), iterated over the full dataset.

Proof. There are two methods to prove this result. First, we can note that for the standard ELBO—constructed from a single application of Jensen’s inequality, exchanging the log and the expectation—the variational lower bound is tight if the chosen variational family contains the true posterior (see e.g. Knoblauch et al. (2019) for discussion). Our chosen variational posterior formulation consists of independent posteriors for each output dimension, with dense covariances. This contains the exact posterior for multivariate Bayesian linear regression with diagonal Σ and thus the variational lower bound is tight under our modeling assumptions. Moreover, because the variational lower bound (for fixed θ) is strictly concave for appropriate choice of priors, the maximizer is unique.

A second proof approach is constructive: note that the variational lower bound is strictly concave and compute the maximizer by computing the gradient and using first order necessary conditions for optimality. By inspection one can see that this is equivalent to the recursive least squares posterior. $\square$

A.2 TRAINING DETAILS

For training the VBLL models, we closely follow Harrison et al. (2024). For all experiments, we use AdamW (Loshchilov & Hutter, 2017) as our optimizer with a learning rate of $1e^{-3}$, set the weight decay for the backbone (*not* including the parameters of the VBLL) to $1e^{-4}$, and use norm-based gradient clipping with a value of 1. For the VBLL, we set the prior scale to 1 and the Wishart scale to 0.01. A sensitivity analysis of the Wishart scale on the performance and run time is in Appendix D. As mentioned in Sec. A.2, we employ early stopping for all VBLL models based on the training loss. We track the average loss of a training epoch and if this average loss does not improve for a 100 epochs in a row, we stop training and use the model parameters that yielded the lowest training loss.

A.3 FURTHER DETAILS ON THE SETUP AND ACQUISITION FUNCTIONS

We implement all baselines and experiment in BoTorch Balandat et al. (2020) and GPyTorch Gardner et al. (2018). As best-practice in BO, we standardize the data to mean zero and a variance of one at each iteration. We further transform the input space specified by the problem into the hypercube $\mathcal{X} \in [0, 1]^D$. For completeness, we again list all the baselines below and then discuss the optimization of the acquisition functions.

Acquisition functions: We use 10 restarts and 512 raw samples for optimizing the acquisition functions UCB and $\log\text{EI}$ for all models. For TS we optimize the analytic sample of the VBLLs with 10 random restart using L-BFGS-B Zhu et al. (1997) as the optimization method. For TS from the non-parametric models, we use the same heuristic as in Eriksson et al. (2024) and generate $\min\{5000, \max\{2000, 200 \cdot D\}\}$ pseudo-random input points from a Sobol sequence and then sample from the high-dimensional multi-variate normal. The next location is then the argmax of the sample path. For the multi-objective problems, we use $\log\text{EHVI}$ for all models with consistent reference points. For TS with VBLLs, we follow the procedure described in Sec. 4. For NSGA-II, we use a population size of 100 and terminate after 200 generations.

B ON NEURAL NETWORK THOMPSON SAMPLING

We further investigated the use of using neural network based Thompson samples. With VBLLs, we effectively maintain a distribution over plausible deterministic NNs that are in accordance with

the current data set and noise level. Instead of maintaining such a distribution and then sampling from the variational posterior of the weights w , a straight-forward idea would be to *directly* train a deterministic neural network using the same optimizer (including the same early stopping etc.), and L2 loss, as well as L2 regularization for the backbone directly generating a MAP Thompson sample. Note that with such an approach, one would no longer be able to leverage acquisition functions that rely on good uncertainty quantification such as $\log EI$ or more sophisticated such as information-theoretic acquisition functions based on entropy search such as MES Wang & Jegelka (2017), which may require large ensembles (up to 100 NNs), making it computationally expensive. Still, we tested this baseline and the results are summarized in Fig. 6.

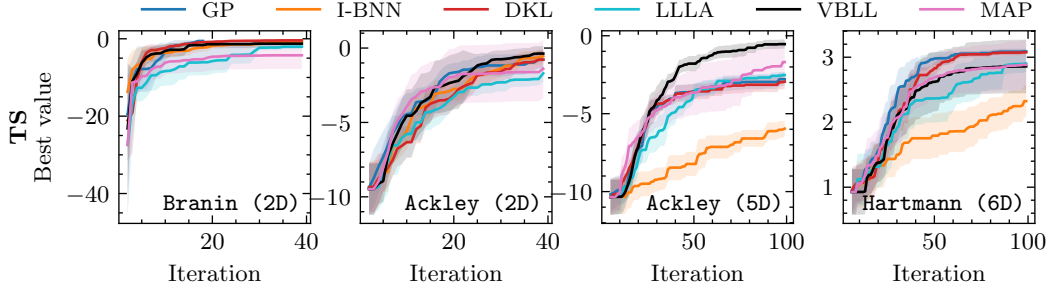


Figure 6: *Comparison of neural network Thompson sampling on synthetic benchmarks.* The deterministic MAP Thompson sample shows surprisingly good performance however also yields large variance on simple benchmarks which is undesirable.

Here MAP refers to the MAP Thompson sample baseline. We can observe that this simple baseline performs surprisingly well but cannot match the performance of the other baselines. We also observe that in relatively simple problems, such as Branin and Ackley2D, the variance is significantly larger compared to the other baselines, which is undesirable in BO applications. We also compared this baseline on the high-dimensional and real-world benchmarks and the results are shown in Fig. 7.

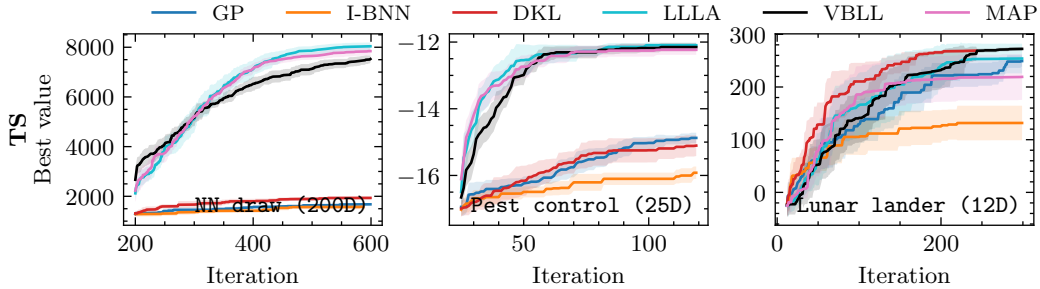


Figure 7: *Comparison of neural network Thompson sampling on high-dimensional and real-world benchmarks.* On these benchmarks, the MAP Thompson sample baseline shows mixed performance. It performs better on NNdraw but exhibits large variance for Lunarlander.

The MAP approach demonstrates superior performance on the NNdraw task. This is likely because the Wishart scale in the VBLL baselines is not well-tuned for the NNdraw problem (cf. Sec.). The data is normalized to zero mean and a standard deviation of one; however, with a Wishart scale greater than zero, the assumed noise in this normalized space affects the accuracy of the correlations between data points—especially for the large value range in NNdraw. In contrast, the MAP baseline, by design, does not account for noise, which may contribute to its better performance in this context. Additionally, the MAP baseline exhibits slightly faster convergence on Pestcontrol. For Lunarlander, the MAP approach shows considerable variance and fails to match the final performance of the VBLLs.

C EXPERIMENTAL DETAILS

Branin: A standard two dimensional optimization benchmark with three global optima.

Ackley: A standard optimization benchmark with various local optima (depending on the dimensionality) and one global optimum. In our experiments, we compare the surrogates on a 2D and 5D version and set the feasible set to the hypercube $\mathcal{X} = [-5, 10]^d$ as in Eriksson et al. (2024).

Hartmann: A standard six dimensional benchmark with six local optima and one global optimum.

NN draw: In this optimization problem, our goal is to find the global optimum of a function defined by a sample from a neural network within the hypercube $\mathcal{X} = [0, 1]^d$. This benchmark was also employed in Li et al. (2024). We use a fully connected neural network with two hidden layers, each containing 50 nodes, and ReLU activation functions. The input size corresponds to the dimensionality of the optimization problem (in our case, 200), and the output size is one. To generate a function, we sample all weights from the standard normal distribution $\mathcal{N}(0, 1)$. For a fair comparison, we use the same fixed seed across all baselines ensuring that the same objective function is used.

Pest control: This optimization problem was also in Li et al. (2024) and aims to minimizing the spread of pests while minimizing the prevention costs of treatment and was introduced in Oh et al. (2019). In this experiment, we define the setting as a categorical optimization problem with 25 categorical variables corresponding to stages of intervention, with 5 different values at each stage. As mentioned in Oh et al. (2019), dynamics behind this problem are highly complex resulting in involved correlations between the inputs.

Lunar lander: Lunar lander is an environment from OpenAI gym. The objective is to maximize the average final reward over 50 randomly generated environments. For this, 12 continuous parameters of a controller have to be tuned as in Eriksson et al. (2024).

Branin Currin: A standard two dimensional benchmark with two objectives.

DTLZ1: A standard benchmark of which we choose a five dimensional version with two objectives.

DTLZ2: A standard benchmark of which we choose a five dimensional version with two objectives.

Oil Sorbent: Building on the implementation by Li et al. (2024), we optimize the properties of a material to maximize its performance as a sorbent for marine oil spills as introduced by Wang et al. (2020). The problem consists of five ordinal parameters and two continuous parameters which control the manufacturing process of electrospun materials, and the three objectives are water contact angle, oil absorption capacity, and mechanical strength.

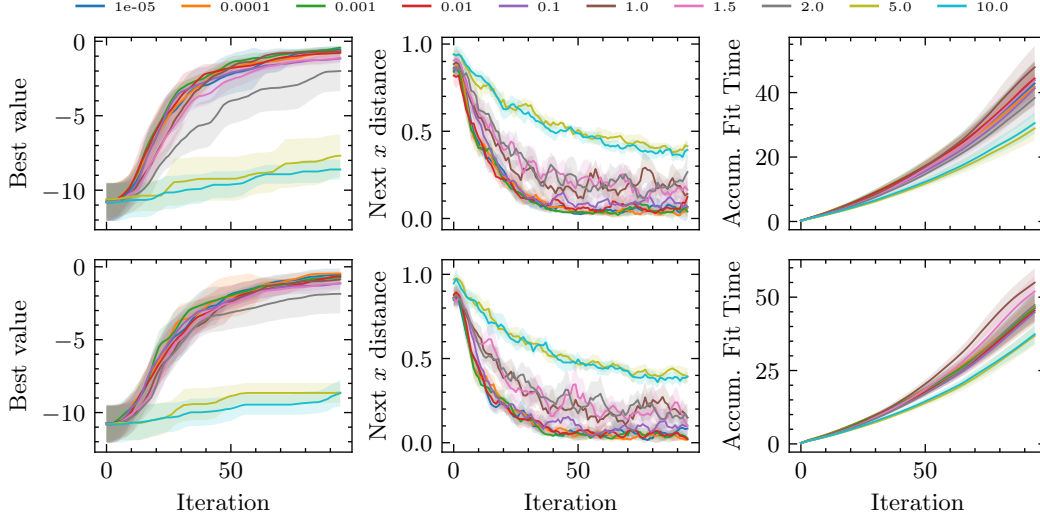
D HYPERPARAMETER SENSITIVITY

The parametric VBLL surrogate has hyperparameters that have to be specified a-priori. In the following, we present results on the hyperparameter sensitivity of the VBLL surrogate model and demonstrate that tuning hyperparameters can improve empirical performance but also that the VBLL surrogate model is rather robust for a wide range of specifications. In Sec. D.1, we will first consider the sensitivity with respect to the Wishart scale and the reinitialization rate for continual learning. Following this, Sec. D.2 then studies the sensitivity regarding the width of the neural network backbone. Lastly, Sec. D.3 considers the robustness to different noise levels.

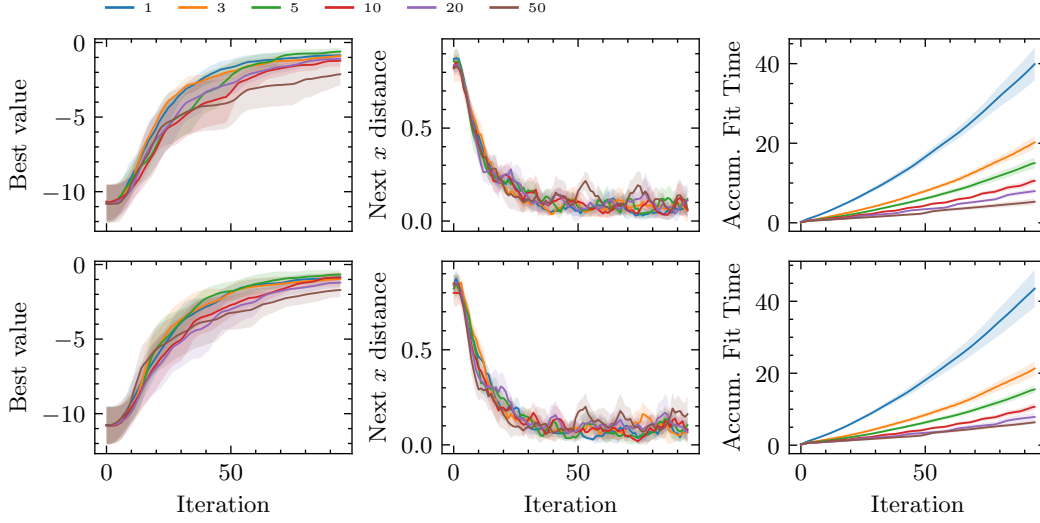
D.1 WISHART SCALE AND CONTINUAL LEARNING SENSITIVITY

We sweep a number of hyperparameters in the VBLLs in order to experiment with the hyperparameter sensitivity of the VBLL models. In particular we sweep the Wishart scale and the re-initialization rate of the model. The re-initialization rate determines how often the VBLL model is re-initialized rather than using CL on the backbone and the variational posterior.

The results of the hyperparameter sweep of the Wishart scale and reinitialization rate on Ackley (5D) can be seen in Fig. 8 and on Pestcontrol in Fig. 9. Please note that we have computed running averages



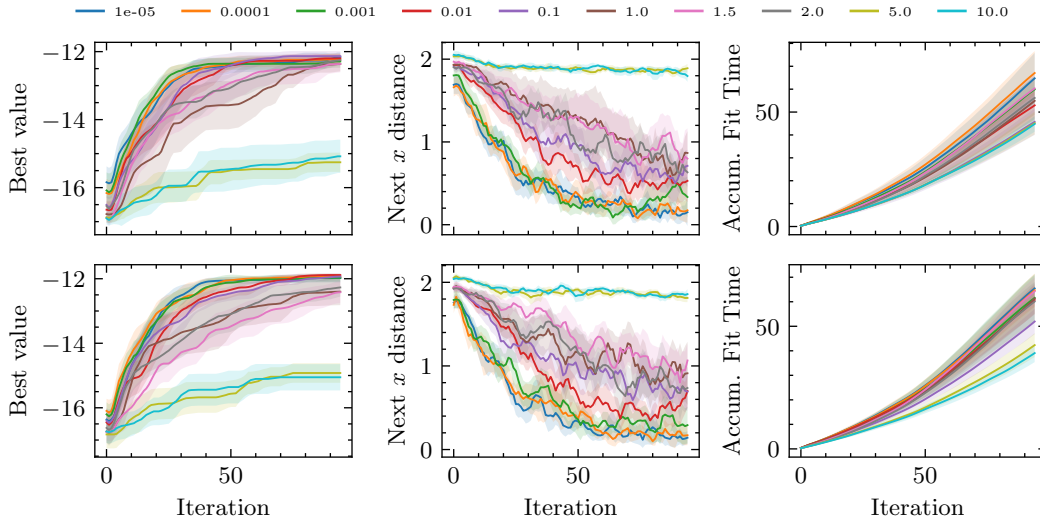
(a) Sensitivity to the Wishart scale (noise free (top), noisy (bottom))



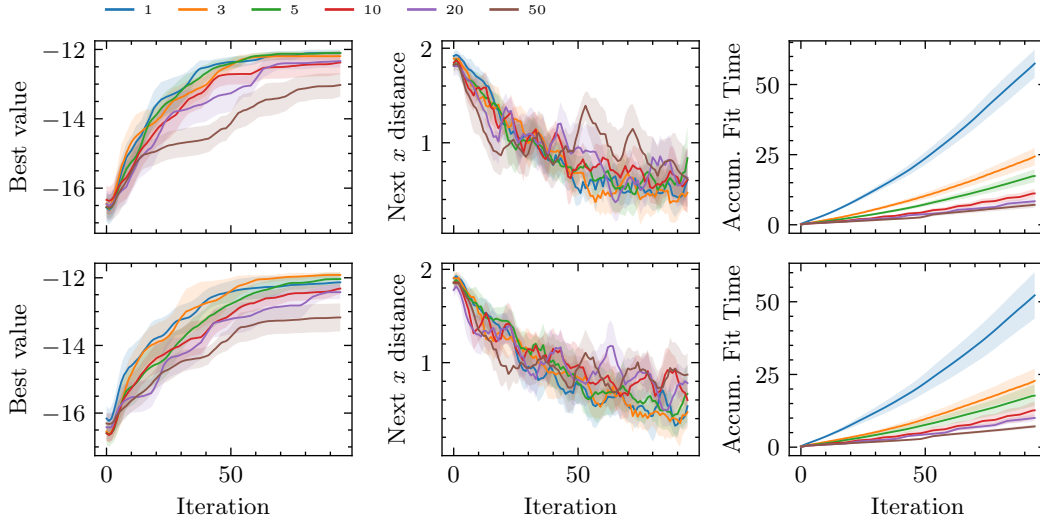
(b) Sensitivity to network reinitialization rate (noise free (top), noisy (bottom))

Figure 8: Hyperparameter sensitivity on the Ackley5D benchmark.

on these figures to make qualitative assessment easier. We find that both of these hyperparameters have impact on BO performance, but that the VBLL models are not exceedingly brittle to the values of these hyperparameters. These results also indicate that continual learning for VBLLs is an area of interest, not only to reduce fitting time, but also because there are indications that the VBLL surrogate benefits from not always being re-initialized (see e.g., Fig. 8 (b) for reinitialization rates 3 and 5). Based on these results we also hypothesise that tuning the Wishart scale appropriately for the problem at hand may lead to increased model performance.



(a) Sensitivity to the Wishart scale (noise free (top), noisy (bottom))



(b) Sensitivity to network reinitialization rate (noise free (top), noisy (bottom))

Figure 9: Hyperparameter sensitivity on the `Pestcontrol` benchmark.

D.2 MODEL WIDTH ABLATION

To evaluate the impact of model width on the performance of both the MAP and VBLL Thompson sampling methods, we conducted a series of experiments on the `Ackley5D` and `Pestcontrol` benchmarks varying the model width.

As illustrated in Fig. 10 (a) and 11 (a), increasing the model capacity (width) of the MAP baseline results in a significant increase in variance, especially pronounced in the `Ackley5D` benchmark. This high variance suggests that the MAP method is highly sensitive to changes in model width, making it challenging to tune effectively for consistent performance across different tasks.

In comparison, the VBLL method exhibits more robustness to model capacity, as shown in Fig. 10 (b) and 11 (b). Despite increasing the model width, VBLL does not suffer from the high variance observed in the MAP baseline. Additionally, the next x distance metric plots in both Fig. 10 and 11 indicates that at later BO stages the VBLL models return to exploring uncertain regions of the input space whereas the MAP models get stuck in local optima accounting for the continuous improvement

in best values for the VBLL models. This robustness is advantageous in practical application where extensive model tuning is unfeasible and hints that VBLL may also perform better when scaling to larger model sizes.

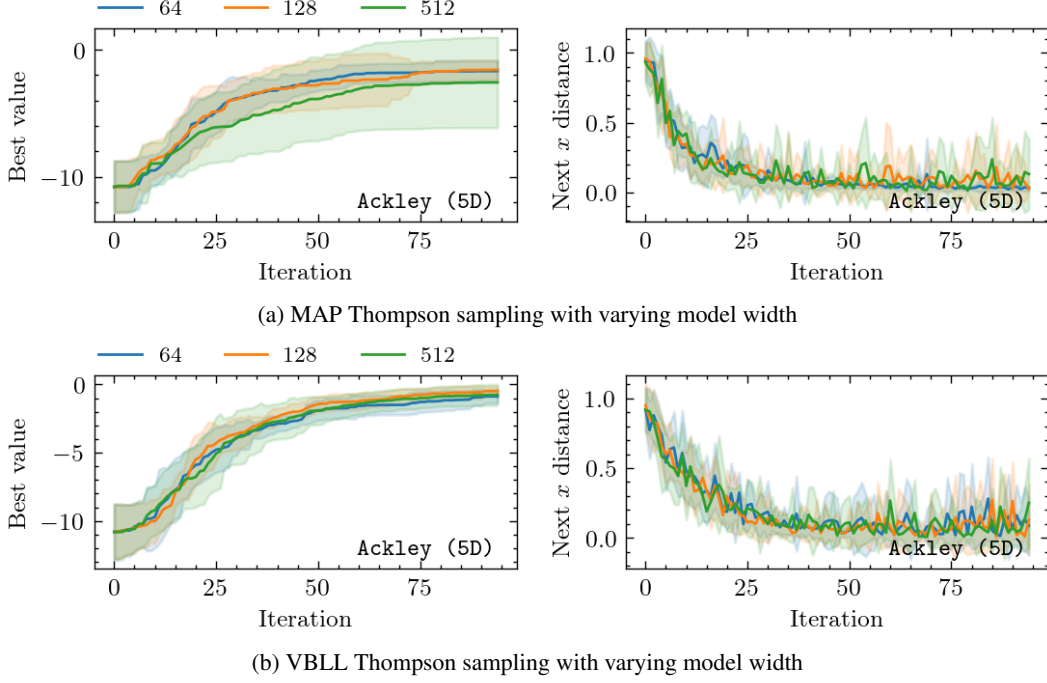


Figure 10: Comparison of neural network Thompson sampling methods on the `Ackley5D` benchmark with varying model width. The models were trained with width of 64, 128 and 512 neurons.

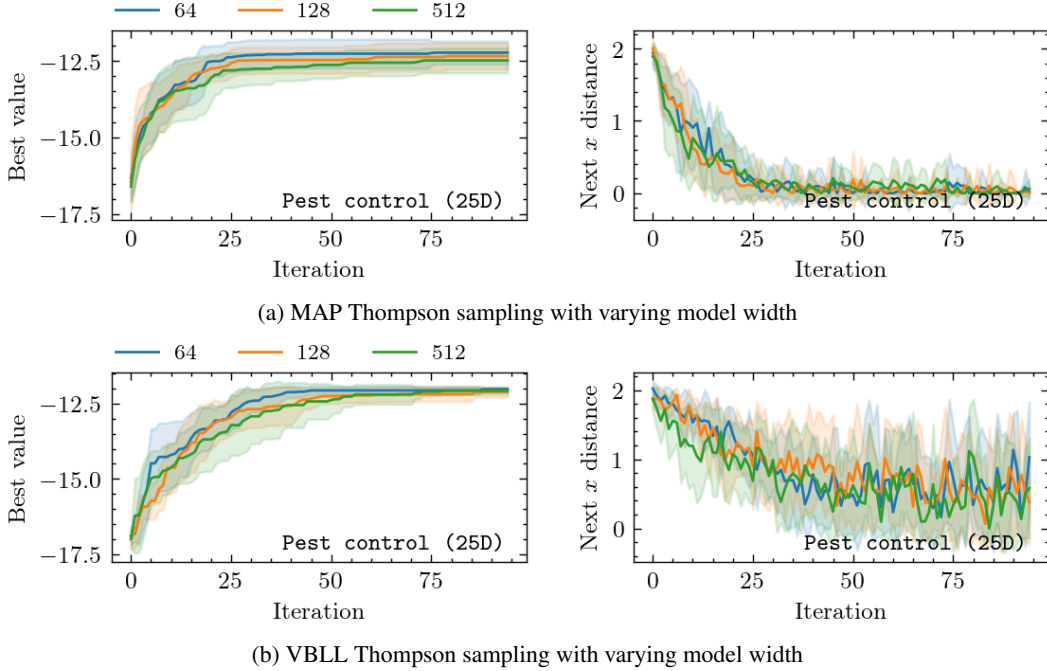
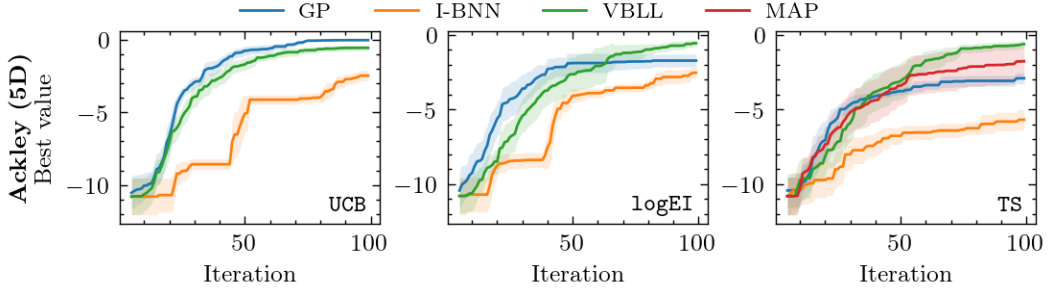


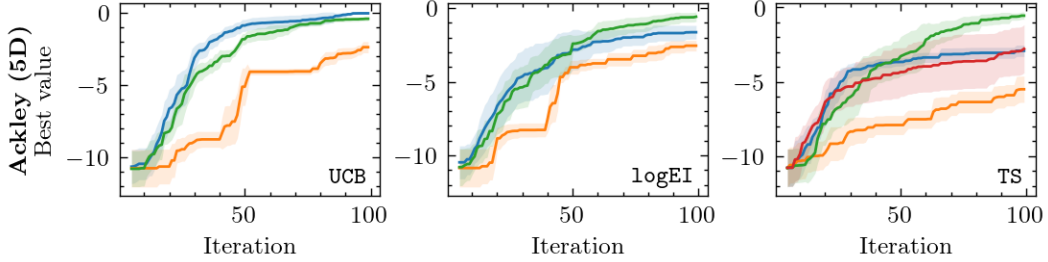
Figure 11: Comparison of neural network Thompson sampling methods on the `Pestcontrol` benchmark with varying model width. The models were trained with width of 64, 128 and 512 neurons.

D.3 MODEL PERFORMANCE IN THE PRESENCE OF NOISE

Lastly, we also benchmark the different surrogates on different noise levels. We again only consider `Ackley5D` (Fig. 12) and `Pestcontrol` (Fig. 13). For these experiments, we use the same Wishart scale of 0.01 for the VBLL baseline. We can observe that all models, besides the MAP baseline in `Ackley5D`, are rather robust the change in noise level.

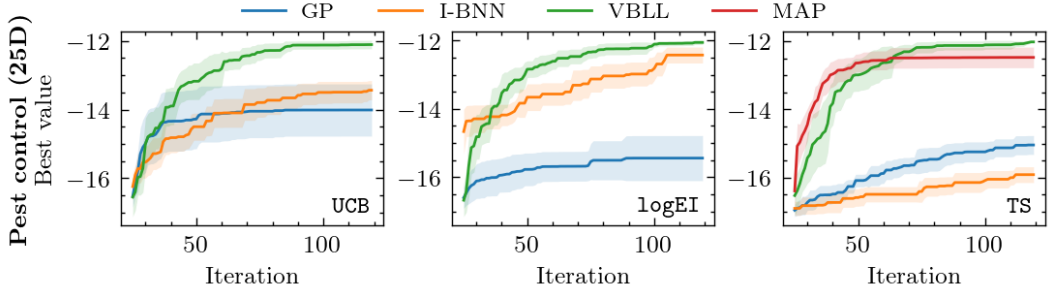


(a) Noisy objective with noise standard deviation of 0.01.

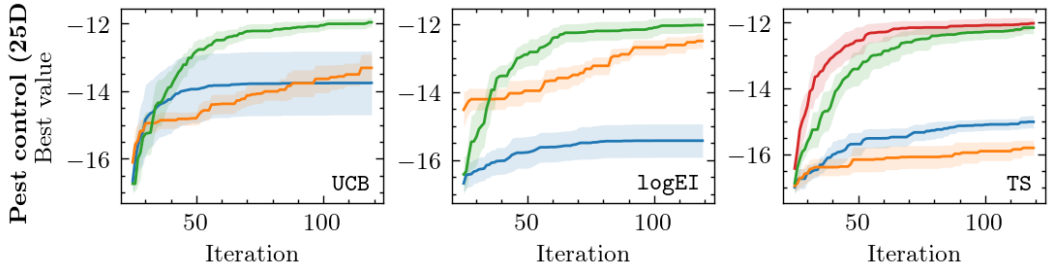


(b) Noisy objective with noise standard deviation of 0.1.

Figure 12: Performance comparison of baseline methods on `Ackley5D` benchmark with noise.



(a) Noisy objective with noise standard deviation of 0.01.



(b) Noisy objective with noise standard deviation of 0.1.

Figure 13: Performance comparison of baseline methods on `Pestcontrol` benchmark with noise.